UNIVERSIDAD DON BOSCO

DISEÑO Y PROGRAMACIÓN DE SOFTWARE MULTIPLATAFORMA

DPS441 G01T

Investigación aplicada 2

Alumno:

- Cesar Alejandro Lara Franco.....LL202677
- Edwin Daniel Leiva Barrera.....LB250368
- Kevin Jafet Aguilar Franco.....AF250113
- Cesar Armando Melendez Arias.....MA250099
- Anderson Alfredo EscobarMoran.....EM251400

Docente:

Emerson Francisco Cartagena Candelario

Fecha de entrega 14/04/2026

Introducción

En la actualidad, el desarrollo de aplicaciones móviles se ha convertido en una de las áreas más importantes dentro de la tecnología, debido al crecimiento del uso de dispositivos móviles en la vida cotidiana. En este contexto, frameworks como React Native han permitido a los desarrolladores crear aplicaciones multiplataforma de manera eficiente, utilizando una sola base de código para sistemas como Android e iOS.

Para enfrentar estos desafíos, es fundamental implementar pruebas de software, las cuales permiten evaluar, verificar y validar el correcto funcionamiento de una aplicación antes de su lanzamiento. Las pruebas no solo ayudan a detectar errores de forma temprana, sino que también contribuyen a mejorar la calidad del código, facilitar su mantenimiento y garantizar la estabilidad del sistema a lo largo del tiempo.

Dentro de este proceso, destacan dos tipos de pruebas esenciales: las pruebas unitarias y las pruebas de integración. Las pruebas unitarias se enfocan en validar el comportamiento de las unidades más pequeñas del sistema de manera aislada, mientras que las pruebas de integración evalúan cómo interactúan diferentes componentes entre sí, simulando escenarios más cercanos al uso real de la aplicación.

En el presente documento se analizarán ambos tipos de pruebas, abordando sus características, beneficios, diferencias y su importancia dentro del desarrollo de aplicaciones React Native, con el fin de comprender cómo contribuyen a la construcción de software más robusto, confiable y de alta calidad.

Índice

Introducción 2
Pruebas Unitarias y Pruebas de Integración en Aplicaciones React Native 4
Pruebas Unitarias 4
- **Beneficios de las pruebas unitarias** 5
- **Frameworks para pruebas unitarias en React Native** 6
- **Ejemplo de prueba unitaria en React Native** 7

Pruebas de Integración 8
- **Beneficios de las pruebas de integración** 8
- **Diferencias entre pruebas unitarias y pruebas de integración** 9
- **Ejemplo de prueba de integración en React Native** 9
- **Frameworks para pruebas de integración en React Native** 10

Análisis comparativo entre pruebas unitarias e integración. 13
Ejemplo de implementación de pruebas unitarias e integración en una aplicación React Native. 15
- **Ejemplo de Integración** 19

Link de GitHub: 23
Link del Video: 23
Capturas del video 24
Conclusiones 25
Bibliografía 26

Pruebas Unitarias y Pruebas de Integración en Aplicaciones React Native

Pruebas Unitarias

Las **pruebas unitarias** son pruebas automáticas que verifican el correcto funcionamiento de la unidad más pequeña de una aplicación. En React Native, una unidad puede ser:

- Una función JavaScript
- Un componente funcional
- Un hook personalizado
- Un módulo de lógica específica

Estas pruebas se realizan **de forma aislada**, es decir, sin depender de otros componentes, servicios externos o APIs reales. El objetivo es comprobar que una unidad produce el resultado esperado ante determinadas entradas.

Ejemplo simple de unidad:

- Una función que suma dos números
- Un componente que muestra un texto según una prop recibida

Beneficios de las pruebas unitarias

Las pruebas unitarias ofrecen múltiples ventajas en el desarrollo de aplicaciones React Native:

1. **Detección temprana de errores**
   Permiten identificar fallos antes de que el código se integre con otras partes del sistema.
2. **Mejor calidad del código**
   Al escribir pruebas, el desarrollador tiende a estructurar mejor su código, creando componentes más claros y reutilizables.
3. **Facilitan el mantenimiento**
   Cuando se realizan cambios o refactorizaciones, las pruebas unitarias ayudan a asegurarse de que no se rompe el comportamiento original.
4. **Documentación implícita**
   Las pruebas muestran cómo debe comportarse un componente o una función, lo que ayuda a otros desarrolladores a entender el sistema.
5. **Mayor confianza al desarrollar**
   El desarrollador puede agregar nuevas funcionalidades con mayor seguridad.

Frameworks para pruebas unitarias en React Native

Jest

Jest es un framework de pruebas en JavaScript utilizado ampliamente para realizar pruebas unitarias en aplicaciones React Native. Este framework permite ejecutar pruebas automatizadas, realizar aserciones y simular dependencias, lo que facilita la verificación del comportamiento de componentes y funciones de forma aislada. React Native incluye Jest de forma predeterminada, lo que lo convierte en la herramienta principal para pruebas unitarias.

Jest destaca por su facilidad de uso, rapidez en la ejecución de pruebas y capacidad para generar resultados claros sobre el estado del código probado.

- Es el framework de pruebas más usado en React Native.
- Viene configurado por defecto al crear un proyecto.
- Permite pruebas unitarias, pruebas de lógica y pruebas de snapshots.
- Ofrece mocks para simular módulos nativos.

React Native Testing Library

React Native Testing Library es una librería que complementa a Jest y está orientada a probar los componentes desde la perspectiva del usuario. Su enfoque principal es verificar lo que se muestra en pantalla y cómo el usuario interactúa con la aplicación, evitando depender de detalles internos de implementación.

Esta librería permite renderizar componentes, buscar elementos visibles y simular eventos como presionar botones o ingresar texto, promoviendo buenas prácticas de prueba.

- Se usa junto con Jest.
- Se enfoca en probar el comportamiento del componente desde el punto de vista del usuario.
- Permite renderizar componentes y simular interacciones como clics o escritura en inputs.

Ejemplo de prueba unitaria en React Native

Componente a probar

```
1  import React from 'react';
2  import { Text } from 'react-native';
3
4  export default function Saludo({ nombre }) {
5    return <Text>Hola {nombre}</Text>;
6  }
7  ``
```

Prueba unitaria con Jest y React Native Testing Library

```
1  import React from 'react';
2  import { render } from '@testing-library/react-native';
3  import Saludo from './Saludo';
4
5  test('muestra el saludo con el nombre recibido', () => {
6    const { getByText } = render(<Saludo nombre="Cesar" />);
7    expect(getByText('Hola Cesar')).toBeTruthy();
8  });
```

En este ejemplo, se verifica que el componente renderice correctamente el texto según la propiedad recibida. La prueba se realiza de forma aislada, sin depender de otros componentes ni servicios externos.

Pruebas de Integración

Las **pruebas de integración** verifican que **varios componentes o módulos funcionen correctamente juntos**. A diferencia de las pruebas unitarias, estas pruebas no evalúan partes aisladas, sino la interacción entre ellas.

En React Native, una prueba de integración puede validar:

- El flujo de navegación entre pantallas
- La interacción entre componentes y el estado global
- Un formulario completo con envío de datos
- La comunicación entre la interfaz y la lógica de negocio

Estas pruebas se acercan más al comportamiento real del usuario dentro de la aplicación.

Beneficios de las pruebas de integración

1. **Validación del flujo completo**
   Permiten comprobar que la aplicación funciona como un todo y no solo por partes.
2. **Reducción de errores en producción**
   Detectan errores que las pruebas unitarias no pueden detectar, como fallos en la navegación o integración entre módulos.
3. **Mayor realismo**
   Simulan comportamientos más cercanos al uso real del usuario.
4. **Confianza antes del despliegue**
   Son ideales para validar funciones críticas como login, registro o pagos.

Diferencias entre pruebas unitarias y pruebas de integración

Mientras que las pruebas unitarias se enfocan en evaluar componentes o funciones de manera aislada, las pruebas de integración verifican el comportamiento conjunto de múltiples componentes dentro de la aplicación. Las pruebas unitarias validan la lógica interna de una sola unidad de código, mientras que las pruebas de integración evalúan flujos completos y la comunicación entre distintas partes del sistema.

En React Native, una prueba unitaria puede verificar que un componente muestre correctamente un texto, mientras que una prueba de integración puede validar que al presionar un botón se navegue correctamente a otra pantalla y se renderice la información esperada. De esta manera, las pruebas de integración permiten detectar errores funcionales que no pueden ser identificados mediante pruebas unitarias.

Ejemplo de prueba de integración en React Native

Un ejemplo de prueba de integración consiste en validar el flujo de inicio de sesión de un usuario. En este escenario, la prueba verifica que el usuario ingrese sus credenciales, presione el botón de inicio de sesión y sea redirigido exitosamente a la pantalla principal de la aplicación.

Este tipo de prueba permite comprobar que los componentes visuales, la lógica de autenticación y la navegación funcionan correctamente de forma conjunta, simulando una situación real de uso por parte del usuario.

Frameworks para pruebas de integración en React Native

Detox

Detox es un framework de pruebas automatizadas diseñado específicamente para realizar pruebas de integración y pruebas End-to-End (E2E) en aplicaciones móviles desarrolladas con React Native.

Fue creado por Wix con el objetivo de solucionar uno de los mayores problemas en las pruebas móviles: la inestabilidad (flakiness) causada por esperas manuales y comportamientos asíncronos no controlados.

Detox permite probar una aplicación tal como la usaría un usuario real, ejecutándola sobre:

- Simuladores iOS
- Emuladores Android
- Dispositivos reales (principalmente Android)

¿Para qué sirve Detox?

Detox se utiliza para validar que:

- La aplicación **arranca correctamente**
- Las pantallas **se muestran como se espera**
- Los botones, formularios y flujos **funcionan de principio a fin**
- La experiencia del usuario es correcta en escenarios reales

Tipo de pruebas que realiza Detox

Detox se utiliza principalmente para:

Pruebas de Integración

Evalúan cómo interactúan las diferentes partes de la aplicación:

- Componentes
- Navegación
- Estado

- Módulos nativos

Pruebas End-to-End (E2E)

Simulan el recorrido completo de un usuario:

1. Abrir la app
2. Iniciar sesión
3. Navegar entre pantallas
4. Realizar acciones (taps, escribir texto, scroll)

Enfoque "Gray-Box Testing"

Uno de los aspectos más importantes de Detox es que utiliza un enfoque llamado **Gray-Box Testing**.

¿Qué significa?

- **Black-box**: el tester no conoce el estado interno de la app
- **White-box**: el tester conoce toda la lógica interna
- **Gray-box (Detox)**: Detox conoce parcialmente el estado interno de React Native

Detox se conecta al **runtime de React Native**, lo que le permite saber:

- Cuándo el JavaScript está "ocupado"
- Cuándo las animaciones han terminado
- Cuándo las llamadas asíncronas se completan

¿Cómo funciona Detox internamente?

El funcionamiento de Detox puede resumirse en estos pasos:

1. La aplicación se compila en modo de prueba
2. Detox lanza la aplicación en un emulador o simulador
3. El test envía instrucciones (tap, type, scroll)
4. Detox **espera automáticamente** a que la app esté lista

5. Se validan resultados visibles en pantalla

Todo esto se controla desde **Jest**, que actúa como test runner.

Appium

Appium es un framework de automatización open-source para pruebas de aplicaciones móviles nativas, híbridas y multiplataforma, incluyendo React Native. Funciona utilizando el protocolo WebDriver, lo que le permite interactuar con la aplicación de forma similar a un usuario real.

Características principales

- Soporta **Android e iOS**
- Permite escribir pruebas en **JavaScript, Java, Python, Ruby**, entre otros
- Funciona sobre **emuladores, simuladores y dispositivos reales**
- No requiere modificar el código fuente de la aplicación [blog.logrocket.com]

Análisis comparativo entre pruebas unitarias e integración.

Las pruebas unitarias y las pruebas de integración constituyen dos enfoques fundamentales dentro del aseguramiento de la calidad del software. Aunque ambas tienen como objetivo validar el correcto funcionamiento de una aplicación, difieren significativamente en su alcance, nivel de abstracción y propósito dentro del ciclo de desarrollo.

Las pruebas unitarias se enfocan en validar las unidades mínimas del sistema de manera aislada. En el contexto de aplicaciones React Native, una unidad puede corresponder a un componente individual, una función o un hook personalizado. El propósito principal de estas pruebas es comprobar que cada componente del sistema se comporte conforme a lo esperado de forma independiente, sin depender de otros módulos o servicios externos. Para lograr este aislamiento, se emplean técnicas como mocks y stubs, lo que permite un control total del entorno de prueba y una ejecución rápida y determinista.

Por otro lado, las pruebas de integración evalúan la interacción entre múltiples unidades del sistema. En lugar de examinar componentes aislados, estas pruebas validan flujos completos de la aplicación, tales como la navegación entre pantallas, la comunicación entre la interfaz de usuario y la lógica de negocio, y la respuesta del sistema ante acciones del usuario. En React Native, las pruebas de integración suelen ejecutarse en emuladores, simuladores o dispositivos reales, lo que proporciona un alto nivel de realismo en la validación del comportamiento del sistema.

Una diferencia clave entre ambos tipos de pruebas es el nivel de realismo. Las pruebas unitarias ofrecen un alto grado de aislamiento, pero un bajo nivel de representación del uso real del sistema. En contraste, las pruebas de integración presentan un mayor costo computacional y complejidad, pero permiten detectar errores que solo aparecen cuando los componentes trabajan conjuntamente.

Asimismo, las pruebas unitarias destacan por su rapidez de ejecución y facilidad de mantenimiento, lo que las convierte en una herramienta ideal para el desarrollo continuo. Las pruebas de integración, aunque más lentas y complejas, resultan esenciales para validar funcionalidades críticas antes del despliegue de la aplicación.

Ambas estrategias no deben considerarse excluyentes, sino complementarias. El uso combinado de pruebas unitarias e integración permite alcanzar un equilibrio adecuado entre precisión técnica y validación funcional, garantizando aplicaciones React Native más robustas, confiables y mantenibles.

Criterio	Pruebas Unitarias	Pruebas de Integración
Foco	Código aislado	Experiencia completa
Aislamiento	Alto	Bajo
Realismo	Bajo	Alto
Velocidad	Muy alta	Media / baja
Complejidad	Baja	Alta
Tipo de errores	Lógicos	Funcionales
Momento de uso	Desarrollo	Preproducción

Ejemplo de implementación de pruebas unitarias e integración en una aplicación React Native.

Aquí presentamos un ejemplo de implementación de prueba unitaria en un componente de **React Native** utilizando la arquitectura de consola (CLI). El objetivo es validar la lógica de negocio de un formulario de forma aislada.

Se ha desarrollado un componente denominado LoginValidator.js que contiene una regla de validación: si la contraseña ingresada por el usuario tiene menos de 6 caracteres, el sistema debe mostrar un mensaje de error. Si cumple con la longitud requerida, el mensaje debe desaparecer.

Este archivo representa la unidad funcional a evaluar. Se utiliza el hook UseState para gestionar la reactividad y la propiedad testID para permitir que el motor de pruebas localice los elementos de la interfaz de forma unívoca.

JS LoginValidator.js > LoginValidator

```
1   import React, { useState } from 'react';
2   import { View, TextInput, Text, StyleSheet } from 'react-native';
3
4   const LoginValidator = () => {
5     // Definimos un estado local para capturar lo que el usuario escribe
6     const [password, setPassword] = useState('');
7
8     return (
9       <View style={styles.container}>
10        {/* El TextInput es nuestro punto de interacción principal */}
11        <TextInput
12          // El testID es como un "ID de identidad" exclusivo para las pruebas.
13          testID="password-input"
14          placeholder="Crea tu contraseña"
15          // Actualizamos el estado cada vez que el usuario teclea
16          onChangeText={(text) => setPassword(text)}
17          secureTextEntry // Oculta los caracteres por seguridad
18        />
19
20        {/* LÓGICA DE NEGOCIO:
21            Si hay texto pero es menor a 6 caracteres, renderizamos el mensaje de error.
22            Esto es lo que vamos a verificar en la prueba unitaria. */}
23        {password.length > 0 && password.length < 6 && (
24          <Text testID="error-message" style={styles.errorText}>
25            La contraseña debe tener al menos 6 caracteres
26          </Text>
27        )}
28      </View>
29    );
30  };
31
32  // Estilos básicos para que el componente no se vea desordenado
33  const styles = StyleSheet.create({
34    container: { padding: 20 },
35    errorText: { color: 'red', marginTop: 5 }
36  });
37
38  export default LoginValidator;
```

Además del componente LoginValidator.js se creo el archivo de prueba LoginValidator.test.js

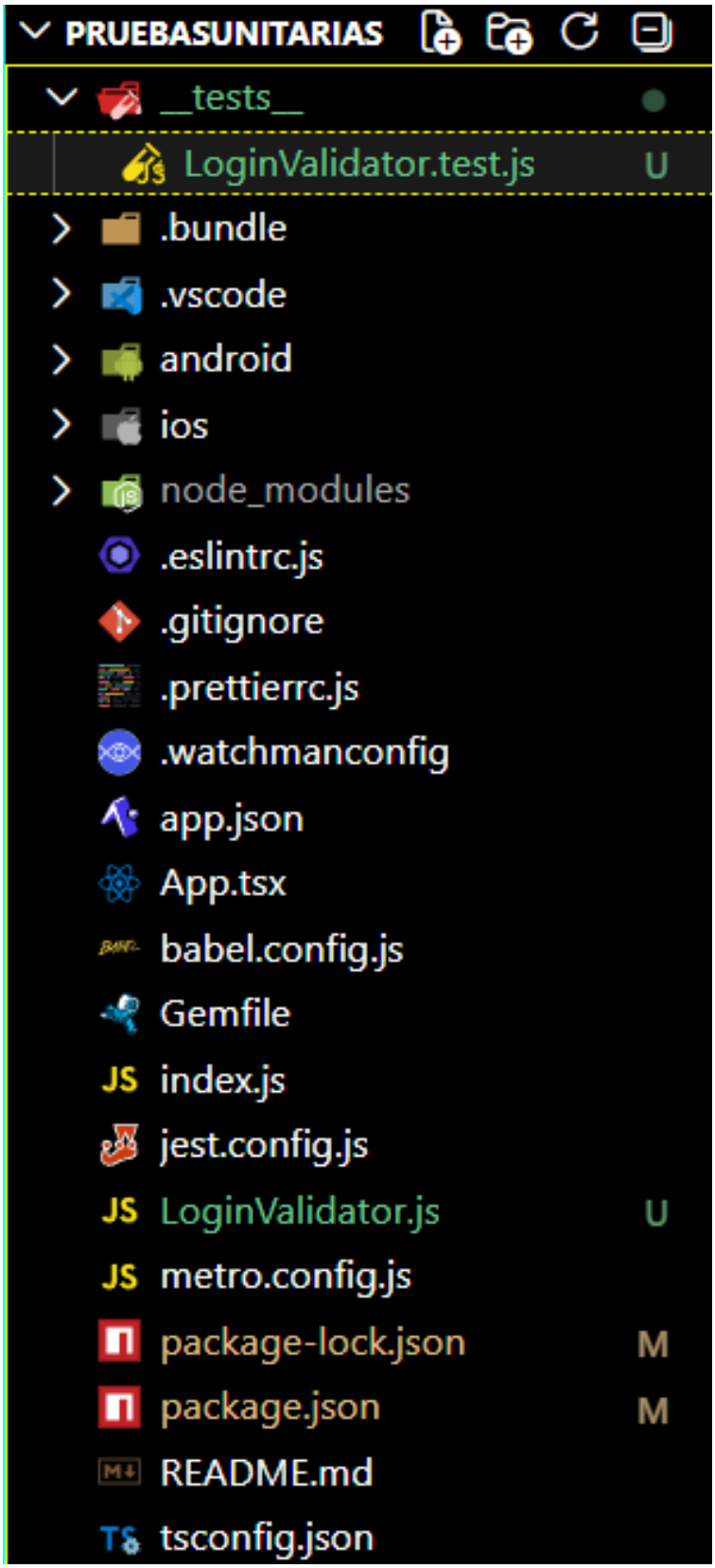
Aquí se estructuro utilizando el patrón AAA (Arrange, Act, Assert) esto para garantizar un proceso de auditoría de código ordenado:

- Configuración (Arrange): Se importo el método render de @testing-library/React-native para montar el componente en un DOM virtual (fuera de un emulador físico).
- Ejecución (Act): Se utiliza fireEvent.changeText para simular la interacción de un usuario real ingresando diferentes cadenas de texto.
- Verificación (Assert): En el caso 1, se valida que el mensaje de error aparezca cuando la entrada es inválida (3 caracteres). En el caso 2, se utiliza queryByTestId para asegurar que el mensaje sea null cuando la entrada es válida (6 caracteres), confirmando que el componente limpia la interfaz correctamente.

```
LoginValidator.test.js U
__tests__ > LoginValidator.test.js > describe('Pruebas Unitarias: Validaciones de Login') callback
1  import React from 'react';
2  // 'render' dibuja el componente en memoria, 'fireEvent' simula acciones humanas
3  import { render, fireEvent } from '@testing-library/react-native';
4  import LoginValidator from '../LoginValidator';
5
6  // 'describe' agrupa nuestras pruebas bajo un título general
   Run | Debug
7  describe('Pruebas Unitarias: Validaciones de Login', () => {
8
9    // CASO 1: Probar que el error aparece cuando la clave es corta
     Run | Debug
10   test('debe mostrar error si la contraseña es demasiado corta', () => {
11     // Renderizamos el componente y extraemos getByTestId para buscar elementos
12     const { getByTestId } = render(<LoginValidator />);
13
14     // Buscamos el input usando el ID que definimos en el componente
15     const input = getByTestId('password-input');
16
17     // ACCIÓN: Simulamos que el usuario escribe "abc" (3 letras)
18     fireEvent.changeText(input, 'abc');
19
20     // ASERCIÓN: Buscamos el mensaje de error por su testID
21     const mensaje = getByTestId('error-message');
22
23     // Verificamos que el texto del error sea exactamente el esperado
24     expect(mensaje.props.children).toBe('La contraseña debe tener al menos 6 caracteres');
25   });
26
27   // CASO 2: Probar que el error NO aparece con datos válidos
     Run | Debug
28   test('no debe mostrar error si la contraseña es válida (6+ caracteres)', () => {
29     // Usamos 'queryByTestId' porque esperamos que algo NO esté en pantalla
30     const { queryByTestId } = render(<LoginValidator />);
31     const input = queryByTestId('password-input');
32
33     // ACCIÓN: Simulamos una contraseña de 6 caracteres
34     fireEvent.changeText(input, '123456');
```

```
35
36     // ASERCIÓN: Intentamos buscar el mensaje de error
37     const mensaje = queryByTestId('error-message');
38
39     // Esperamos que sea 'null' (que no se renderice nada)
40     expect(mensaje).toBeNull();
41   });
42 });
```

Estructura del ejemplo de implementación



Para validar la implementación, se ejecuto el motor de pruebas Jest desde la terminal de visual studio code mediante el comando npm test -- --verbose.

Test suites: Nos indica que el archivo de pruebas fue procesado íntegramente.

Tests: Confirma que los dos escenarios definidos (error y éxito) pasaron satisfactoriamente las aserciones.

Tiempo de ejecución: La prueba se completó en un entorno virtual en menos de 5 segundos, lo que demuestra la eficiencia de las pruebas unitarias frente a las pruebas manuales en dispositivos físicos.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    QUERY RESULTS

● PS C:\EjemploImplementacionPU\PruebasUnitarias> npm test -- --verbose

  > PruebasUnitarias@0.0.1 test
  > jest --verbose

  PASS  __tests__/LoginValidator.test.js
    Pruebas Unitarias: Validaciones de Login
      √ debe mostrar error si la contraseña es demasiado corta (333 ms)
      √ no debe mostrar error si la contraseña es válida (6+ caracteres) (12 ms)

  Test Suites: 1 passed, 1 total
  Tests:       2 passed, 2 total
  Snapshots:   0 total
  Time:        2.003 s, estimated 4 s
  Ran all test suites.
```

Aqui los dos test pasaron ya que cumplen los dos casos.

```
PS C:\EjemploImplementacionPU\PruebasUnitarias> npm test -- --verbose

 > PruebasUnitarias@0.0.1 test
 > jest --verbose

 FAIL  __tests__/LoginValidator.test.js
   Pruebas Unitarias: Validaciones de Login
     × debe mostrar error si la contraseña es demasiado corta (316 ms)
     √ no debe mostrar error si la contraseña es válida (6+ caracteres) (11 ms)

   ● Pruebas Unitarias: Validaciones de Login › debe mostrar error si la contraseña es demasiado corta

     expect(received).toBe(expected) // Object.is equality

     Expected: "Este texto es diferente"
     Received: "La contraseña debe tener al menos 6 caracteres"

       22 |
       23 |     // Verificamos que el texto del error sea exactamente el esperado
     > 24 |     expect(mensaje.props.children).toBe('Este texto es diferente');
          |                                    ^
       25 |   });
       26 |
       27 |   // CASO 2: Probar que el error NO aparece con datos válidos

       at Object.toBe (__tests__/LoginValidator.test.js:24:36)

 Test Suites: 1 failed, 1 total
 Tests:       1 failed, 1 passed, 2 total
 Snapshots:   0 total
 Time:        2.878 s
 Ran all test suites.
```

En este caso nos dio error ya que la contraseña es demasiado corta.

Ejemplo de Integración

Este archivo de prueba en React Native Testing Library valida el comportamiento de un formulario de login.

Prueba con credenciales correctas:

- Renderiza el formulario.
- Simula que el usuario escribe correo y contraseña válidos.
- Presiona el botón de enviar.
- Verifica que aparece un mensaje de bienvenida en la interfaz.

Prueba con credenciales incorrectas:

- Renderiza el formulario.
- Simula que el usuario escribe un correo inválido.
- Presiona el botón de enviar.
- Comprueba que no aparece el mensaje de bienvenida.

```
JS FormularioLogin.js    FormularioLogin.test.js ×
PruebaIntegracion > __tests__ > FormularioLogin.test.js > ...
1  import React from 'react';
2  import { render, fireEvent, screen } from '@testing-library/react-native';
3  import FormularioLogin from '../FormularioLogin';
4
5  describe('Pruebas de Integración - Formulario de Login', () => {
6
7    test('debe permitir al usuario escribir credenciales y ver el mensaje de éxito', () => {
8      // 1. Renderizar el componente completo
9      render(<FormularioLogin />);
10
11     // 2. Buscar los elementos por su testID
12     const entradaCorreo = screen.getByTestId('entrada-correo');
13     const entradaContrasena = screen.getByTestId('entrada-contrasena');
14     const botonEnviar = screen.getByTestId('boton-enviar');
15
16     // 3. Simular la interacción del usuario (Entrada de datos)
17     fireEvent.changeText(entradaCorreo, 'admin@ejemplo.com');
18     fireEvent.changeText(entradaContrasena, '123456');
19
20     // 4. Simular el clic en el botón
21     fireEvent.press(botonEnviar);
22
23     // 5. Verificar el resultado de la integración (Cambio en la UI)
24     const mensajeBienvenida = screen.getByTestId('mensaje-bienvenida');
25
26     expect(mensajeBienvenida).toBeTruthy();
27     expect(mensajeBienvenida.props.children).toBe('¡Bienvenido al Sistema!');
28   });
29
30   test('no debe mostrar el mensaje de bienvenida con credenciales incorrectas', () => {
31     render(<FormularioLogin />);
32
33     const entradaCorreo = screen.getByTestId('entrada-correo');
34     const botonEnviar = screen.getByTestId('boton-enviar');
35
36     fireEvent.changeText(entradaCorreo, 'usuario@error.com');
37     fireEvent.press(botonEnviar);
38
39     // Verificamos que el elemento NO existe en la pantalla
40     const mensaje = screen.queryByTestId('mensaje-bienvenida');
41     expect(mensaje).toBeNull();
42   });
43 });
```

Este componente en React Native es un formulario de inicio de sesión sencillo.

Lo que hace es lo siguiente:

- Muestra dos campos de texto: uno para correo electrónico y otro para contraseña.
- El usuario escribe sus credenciales en esos campos.

- Al presionar el botón “Iniciar Sesión”, se ejecuta una función que valida los datos.
- Si el correo es admin@ejemplo.com y la contraseña es 123456, se activa un estado que muestra un mensaje de éxito.
- Ese mensaje es “¡Bienvenido al Sistema!”, y solo aparece cuando las credenciales son correctas.
- Si los datos no coinciden, el mensaje no se muestra.

JS FormularioLogin.js × FormularioLogin.test.js

PruebaIntegracion > JS FormularioLogin.js > default

```javascript
import React, { useState } from 'react';
import { View, Text, TextInput, Button } from 'react-native';

const FormularioLogin = () => {
  const [correo, setCorreo] = useState('');
  const [contrasena, setContrasena] = useState('');
  const [mensajeExito, setMensajeExito] = useState(false);

  const manejarEnvio = () => {
    if (correo === 'admin@ejemplo.com' && contrasena === '123456') {
      setMensajeExito(true);
    }
  };

  return (
    <View>
      <Text>Correo Electrónico</Text>
      <TextInput
        placeholder="Ingresa tu correo"
        value={correo}
        onChangeText={setCorreo}
        testID="entrada-correo"
      />

      <Text>Contraseña</Text>
      <TextInput
        placeholder="Ingresa tu contraseña"
        value={contrasena}
        onChangeText={setContrasena}
        secureTextEntry
        testID="entrada-contrasena"
      />

      <Button title="Iniciar Sesión" onPress={manejarEnvio} testID="boton-enviar" />

      {mensajeExito && (
        <Text testID="mensaje-bienvenida">¡Bienvenido al Sistema!</Text>
      )}
    </View>
  );
};

export default FormularioLogin;
```

Ese resultado de las pruebas indica lo siguiente:

El archivo de pruebas de la aplicación principal pasó bien: la app se renderizó correctamente.

En el archivo de pruebas del formulario de login.

La prueba que verificaba que no se muestre el mensaje con credenciales incorrectas pasó. Con datos inválidos, el mensaje de bienvenida no apareció, lo cual es correcto y causa el error que no se comuniquen los archivos o fallen.

```
PS C:\Users\ander\OneDrive\Desktop\Investigacion-Aplicada2-DPSLaboratorio\PruebaIntegracion> npm test -- --verbose

> PruebaIntegracion@0.0.1 test
> jest --verbose

 PASS  __tests__/App.test.tsx
  √ renders correctly (123 ms)

 FAIL  __tests__/FormularioLogin.test.js
  Pruebas de Integración - Formulario de Login
    × debe permitir al usuario escribir credenciales y ver el mensaje de éxito (850 ms)
    √ no debe mostrar el mensaje de bienvenida con credenciales incorrectas (22 ms)
```

Ese resultado muestra que todas las pruebas se ejecutaron correctamente:

El archivo de pruebas de la aplicación principal pasó bien: la app se renderizó sin problemas.

El archivo de pruebas del formulario de login también pasó:

La prueba con credenciales correctas mostró el mensaje de bienvenida como se esperaba.

La prueba con credenciales incorrectas confirmó que el mensaje no aparece, lo cual es correcto.

```
PS C:\Users\ander\OneDrive\Desktop\Investigacion-Aplicada2-DPSLaboratorio\PruebaIntegracion> npm test -- --verbose

> PruebaIntegracion@0.0.1 test
> jest --verbose

 PASS  __tests__/App.test.tsx
  √ renders correctly (129 ms)

 PASS  __tests__/FormularioLogin.test.js
  Pruebas de Integración - Formulario de Login
    √ debe permitir al usuario escribir credenciales y ver el mensaje de éxito (1029 ms)
    √ no debe mostrar el mensaje de bienvenida con credenciales incorrectas (11 ms)

Test Suites: 2 passed, 2 total
Tests:       3 passed, 3 total
Snapshots:   0 total
Time:        3.732 s, estimated 4 s
Ran all test suites.
```

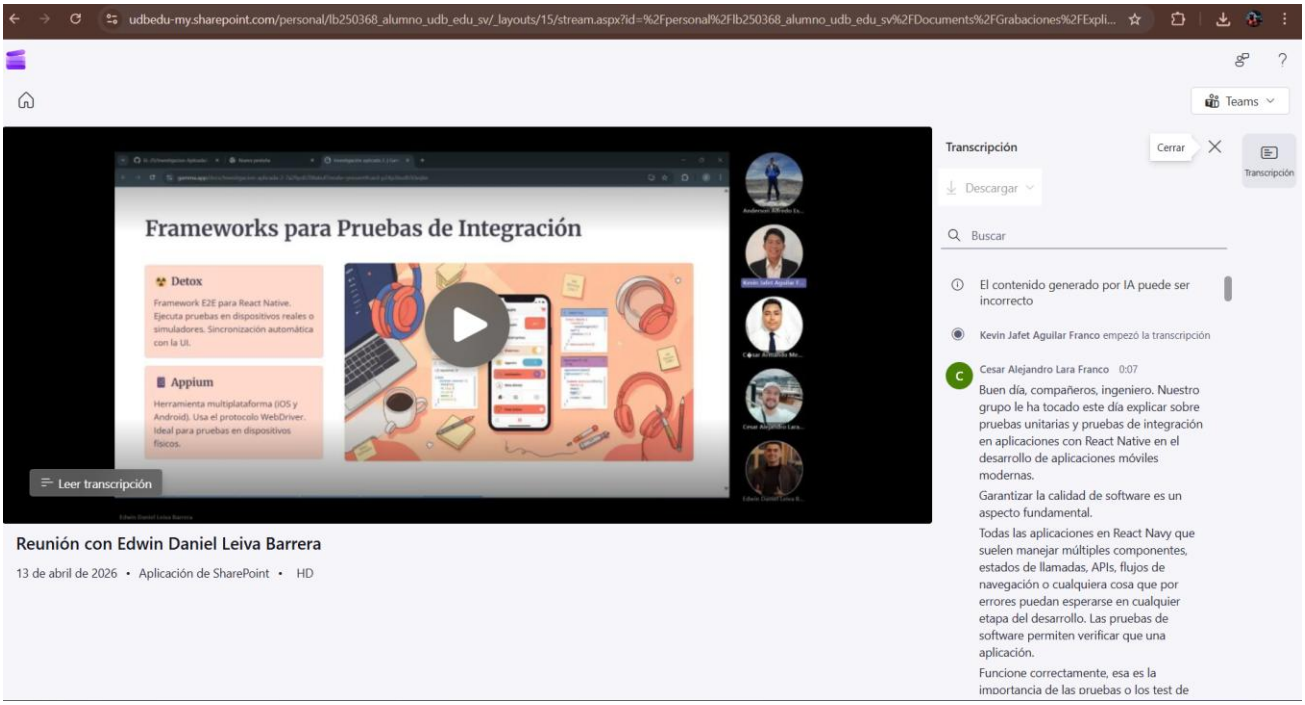
Link de GitHub:

https://github.com/EL-25/Investigacion-Aplicada2-DPSLaboratorio.git

Link del Video:

https://udbedu-my.sharepoint.com/:v:/g/personal/lb250368_alumno_udb_edu_sv/IQBSZWUrYX8FQY4dIuMXt5KNAdKlY0pUCYSnSlHiC1ZwiFU?e=uxyQxJ&nav=eyJyZWZlcnJhbEluZm8iOnsicmVmZXJyYWxBcHAiOiJTdHJlYW1XZWJBcHAiLCJyZWZlcnJhbFZpZXciOiJTaGFyZURpYWxvZy1MaW5rIiwicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldlYiIsInJlZmVycmFsTW9kZSI6InZpZXcifX0%3D

Capturas del video



Conclusiones

En conclusión, las pruebas de software desempeñan un papel fundamental en el desarrollo de aplicaciones móviles modernas, especialmente en entornos como React Native, donde la complejidad de las aplicaciones requiere un alto nivel de control y validación. A lo largo de este análisis, se ha evidenciado que tanto las pruebas unitarias como las pruebas de integración son herramientas esenciales para garantizar el correcto funcionamiento del software.

Las pruebas unitarias permiten validar de forma precisa y aislada cada componente, función o módulo del sistema, facilitando la detección temprana de errores y promoviendo la creación de código más organizado, reutilizable y fácil de mantener. Por su parte, las pruebas de integración permiten evaluar el comportamiento conjunto de los diferentes elementos de la aplicación, asegurando que la interacción entre ellos sea correcta y que los flujos principales funcionen como se espera en escenarios reales.

Aunque cada tipo de prueba tiene características, ventajas y limitaciones propias, su verdadero valor radica en su uso combinado. Mientras las pruebas unitarias garantizan la solidez de las partes individuales, las pruebas de integración aseguran la coherencia y funcionalidad del sistema en su totalidad. Esta complementariedad permite reducir significativamente los errores en producción y aumentar la confianza tanto del desarrollador como del usuario final.

Además, la implementación de estas pruebas desde las etapas iniciales del desarrollo no solo mejora la calidad del producto final, sino que también optimiza los tiempos de trabajo, facilita la colaboración entre equipos y reduce costos a largo plazo. En un entorno tecnológico cada vez más competitivo, desarrollar aplicaciones confiables, estables y eficientes se convierte en una necesidad, y el uso adecuado de pruebas de software es una de las mejores estrategias para lograrlo.

En definitiva, adoptar una cultura de pruebas dentro del desarrollo de aplicaciones React Native no solo representa una buena práctica, sino un requisito indispensable para construir soluciones modernas que respondan de manera efectiva a las exigencias del mercado y a las expectativas de los usuarios.

Bibliografía

- Facebook, Inc. (s. f.). *Testing overview*. React Native Documentation. https://reactnative.dev/docs/testing-overview

- Jest Community. (s. f.). *Testing React Native apps*. Jest Documentation. https://jestjs.io/docs/tutorial-react-native

- Callstack. (s. f.). *React Native Testing Library documentation*. https://testing-library.com/docs/react-native-testing-library/intro

- Wix Engineering. (s. f.). *Detox: End-to-end testing and automation framework for mobile apps*. https://wix.github.io/Detox/

- Suriano, E. (2023). *React Native end-to-end testing with Detox*. LogRocket Blog. https://blog.logrocket.com/react-native-end-to-end-testing-detox/

- Sharma, P. (2023). *Unit testing in React Native using Jest and React Testing Library*. Medium. https://medium.com/@inboxofpulkit/unit-testing-in-react-native-a-comprehensive-guide-with-jest-and-react-testing-library-dc97826a39d6

- Kan, Y. (2026). *Detox: Gray-box testing for React Native applications*. https://yrkan.com/blog/detox-react-native-grey-box/